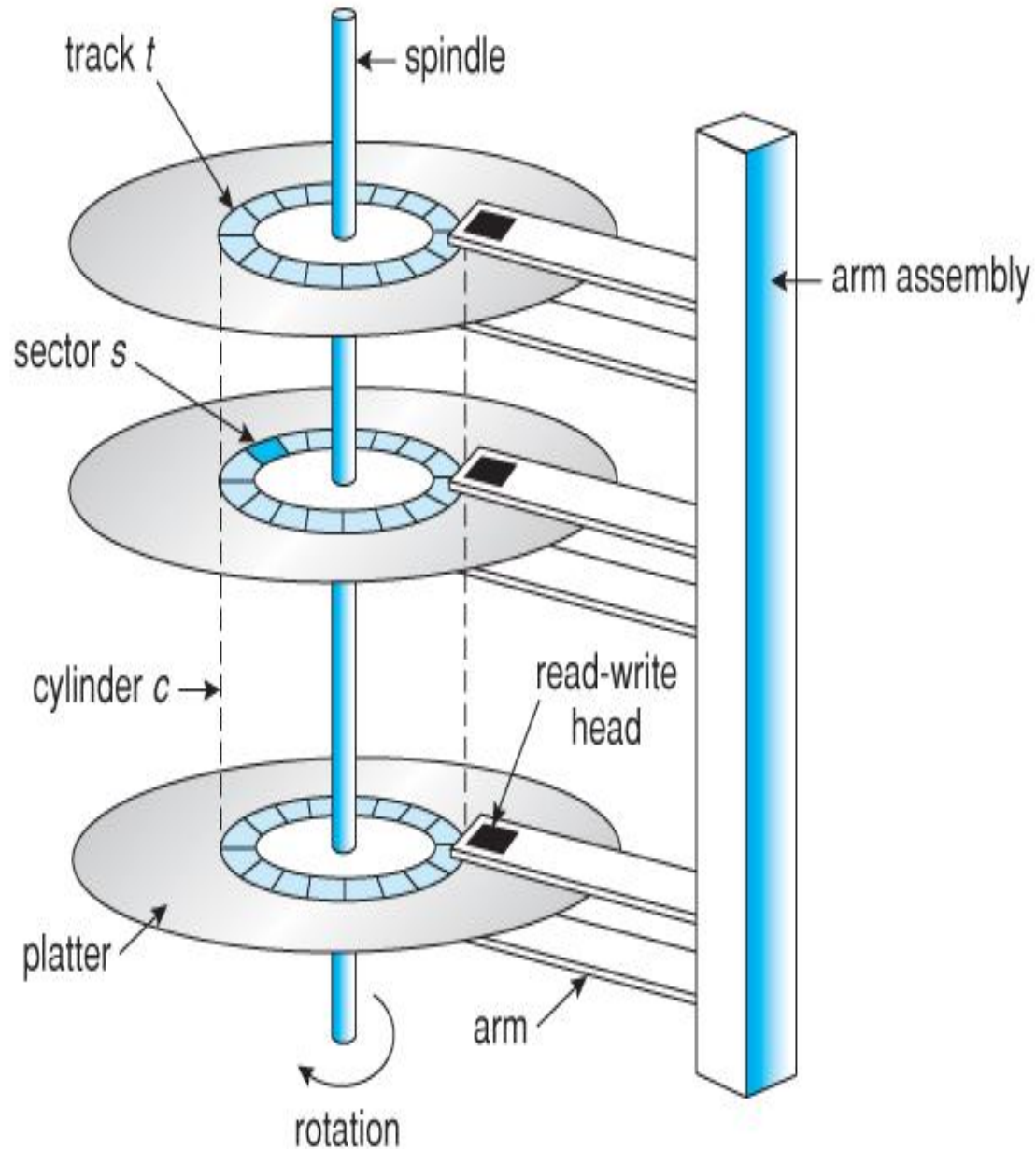




- In operation the disk rotates at high speed, such as 7200 rpm (120 revolutions per second.) The rate at which data can be transferred from the disk to the computer is composed of several steps: The **positioning time**, a.k.a. the **seek time** or **random access time** is the time required to move the heads from one cylinder to another, and for the heads to settle down after the move. This is typically the slowest step in the process and the predominant bottleneck to overall transfer rates.

- The **rotational latency** is the amount of time required for the desired sector to rotate around and come under the read-write head. This can range anywhere from zero to one full revolution, and on the average will equal one-half revolution. This is another physical step and is usually the second slowest step behind seek time. (For a disk rotating at 7200 rpm, the average rotational latency would be $\frac{1}{2}$ revolution / 120 revolutions per second, or just over 4 milliseconds, a long time by computer standards.

- The **transfer rate**, which is the time required to move the data electronically from the disk to the computer. (Some authors may also use the term transfer rate to refer to the overall transfer rate, including seek time and rotational latency as well as the electronic data transfer rate.)

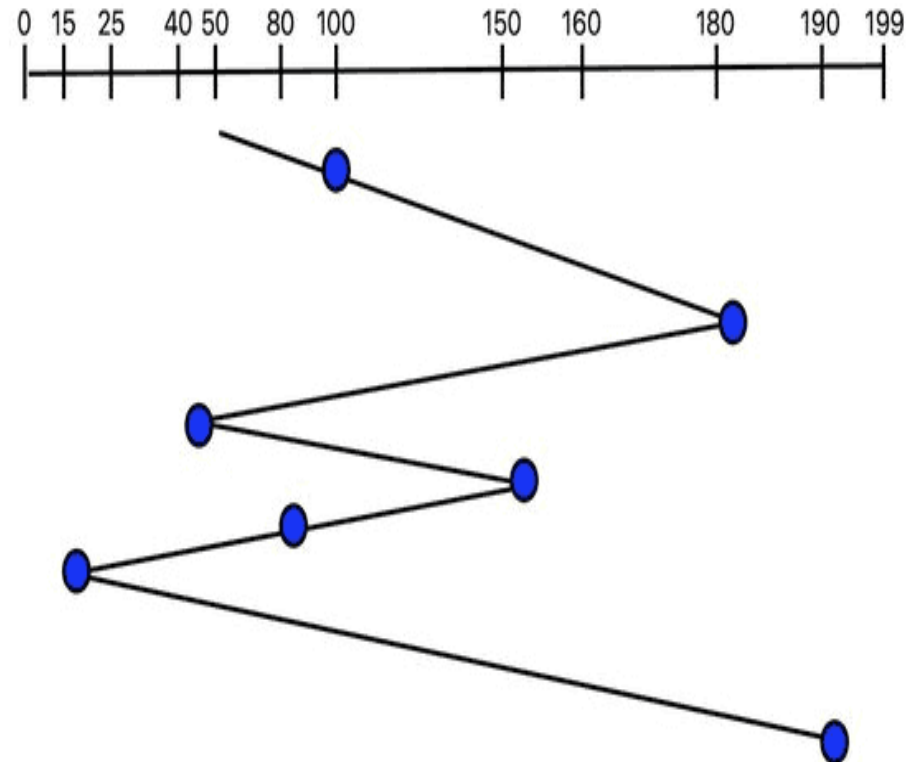
What Are Disk Scheduling Algorithms?

- Disk Scheduling Algorithms refer to the particular manner in which the operating system schedules I/O requests arriving to be serviced by the disk.
- It is important to define these scheduling algorithms to avoid having a clustered hard drive and to efficiently access data. For instance, having a defined service sequence for requests defeats the need of unnecessarily waiting in the queue. The major purpose of implementing disk scheduling algorithms is to reduce the seek time.
- Seek Time refers to the time taken for the disk controller to locate a piece of stored data. The seek time can vary based on the type of scheduling algorithm used and the distance from the start point to where the read/write head of the disk is. Now let's compare disk scheduling algorithms.

Types of Disk Scheduling Algorithms

1. First Come First Serve (FCFS)

- As the name implies, the [First Come First Serve](#) scheduling algorithm services requests in order of their arrival. That is, the request that arrives first is serviced first, and then subsequent requests are also placed in a queue in order of their arrival. Consider the FCFS disk scheduling algorithm example below:



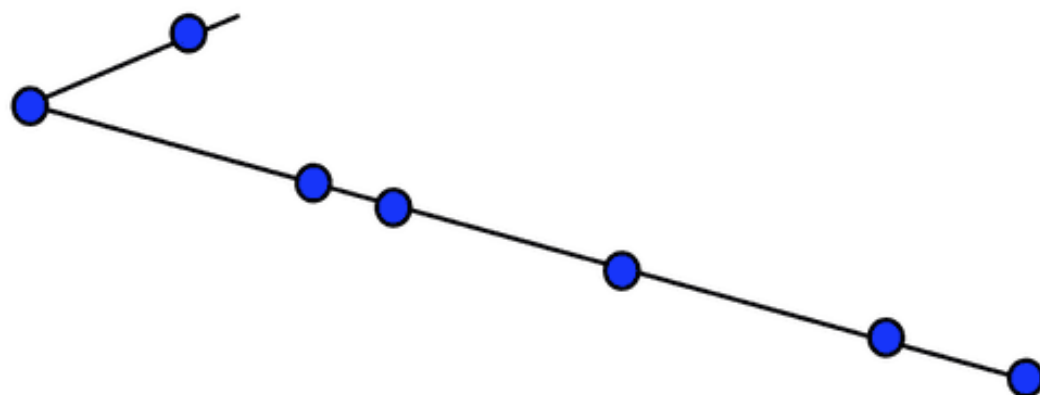
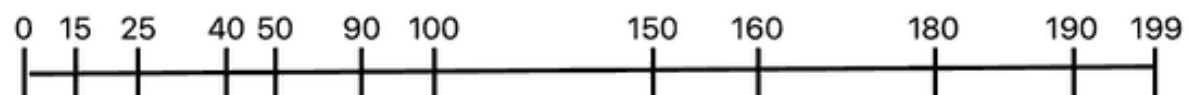
- Here, the requests arrive in this order: (100,180,40,150,80,15,190).
- To find the total head movements, we calculate the number of tracks it took from one request to another.
- Hence, the total head movements for the FCFS algorithm above is given by $= (100-50)+(180-100)+(180-40)+(150-40)+(150-80)+(80-15)+(190-15) = 690$.

Advantages of FCFS

- It is the simplest disk scheduling algorithm and easy to implement.
- Since all requests are serviced in order of their arrival, there is no starvation of any kind. [Starvation](#) is a problem where a process does not get executed due to a lack of the needed resources. It occurs when high-priority processes continually get executed thereby causing low-priority processes to wait indefinitely.
- All requests are treated fairly and eventually get attended to.

Shortest Seek Time First (SSTF)

- The Shortest Seek Time First disk scheduling algorithm processes requests based on the length of their seek time. That is, the request with the shortest seek time gets serviced first. In the execution of the SSTF algorithm, the disk controller performs a quick scan of all the requests to be serviced and arranges them for processing in order of their seek time. The SSTF disk scheduling algorithm addresses the limitations of the FCFS by prioritizing the reduction in total seek time. The disk controller then services the request with the shortest seek time from its current position.
- Suppose the requests arrive in this order: (100,180,40,150,90,15,190). Here's how the disk controller processes these requests:



Shortest Seek Time First

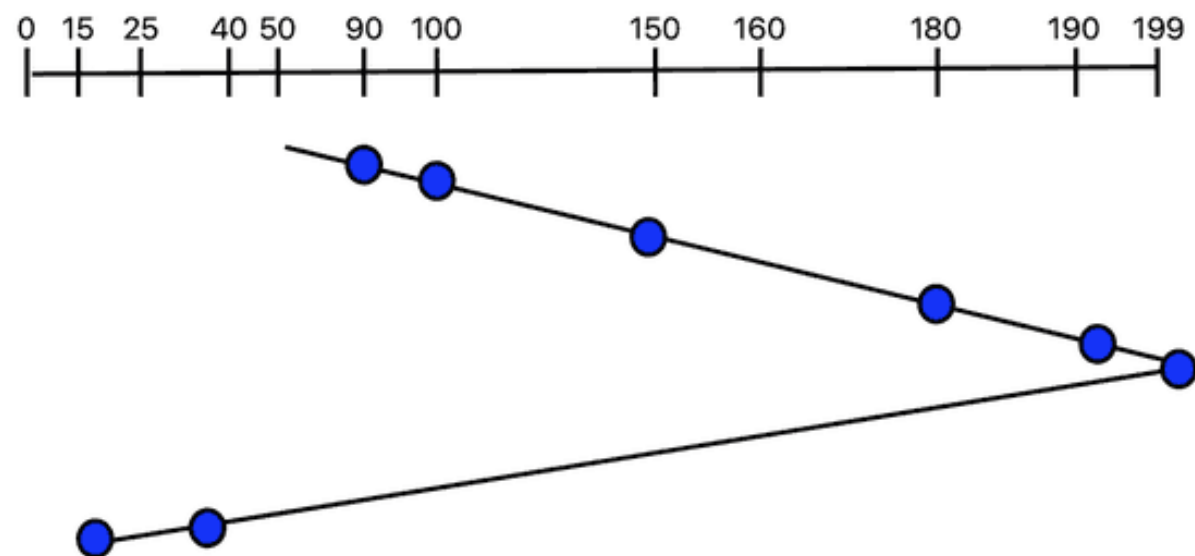
Pieces For Developers

- In the sample diagram above, we see that from the disk head (50), the shortest next distance from the disk head is 40. Note that the request in track 100 arrives first but is not executed first; the request in track 40 is only 10 tracks away from the disk head, which is the shortest and ultimately how the SSTF algorithm works. From track 40, it moves to track 15, which is the shortest next distance being only 25 tracks away and then it progresses that way till it gets to the last track.
- The total head movement is given by $= (50-40)+(40-15)+(90-15)+(100-90)+(150-100)+(180-150)+(190-180)=210$.
- We see that this is a significantly reduced average seek time in comparison to the FCFS algorithms.

- Advantages of SSTF
- It has a significantly reduced average seek time.
- It provides increased throughput. Throughput refers to the number of processes completed per unit time.
- Disadvantages of SSTF
- One major setback of using the SSTF algorithm is the risk of starvation of a request with a higher seek time compared to other requests.
- There is a high variance in response time and waiting time. Response time refers to the amount of time it takes the disk to **respond** to a particular request while waiting time refers to the total amount of time a request spends in the ready queue before reaching the disk.
- There is an overhead in finding the closest request since it has to calculate the seek time of all requests in advance.

SCAN Algorithm

- This is also known as the elevator algorithm because of its working principle. The SCAN disk scheduling algorithm scans down in a single direction to the nearest end from the disk head and then reverses its direction and services all the other requests in the opposite direction.
- Due to the working principle of the SCAN algorithm, requests placed at the midrange position from the disk head have a higher chance of getting serviced than those at the end of the disk head and definitely a higher chance than requests placed behind the disk head. Let's consider an example where our requests arrive in this order: (100,180,40,150,90,15,190). Using the SCAN algorithm in disk scheduling, the requests are serviced in this manner:



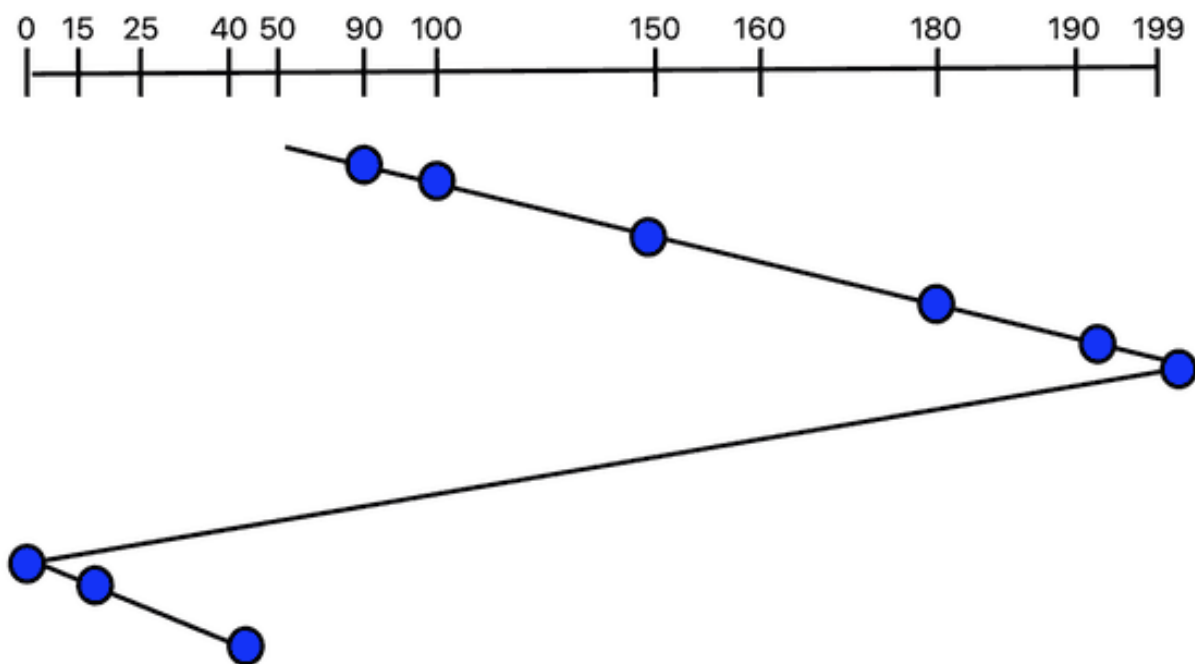
SCAN Algorithm

Pieces For Developers

- Here, we see the requests to the left end are initially serviced, and then it reverses its direction (as we discussed) to service requests on track 40 and track 15.
- The total overhead movement is given by: $(90-50)+(100-90)+(150-100)+(180-150)+(190-180)+(199-190)+(199-40)+(40-15)=333$. Alternatively, we can calculate the overhead movement by subtracting the tracks of the disk head(50) and final point(15) from the tail track(199) where it reverses its direction. We have: $(199-50)+(199-15)=333$
- Advantages of SCAN Algorithm
- It is easy to understand.
- The SCAN algorithm provides a high throughput.
- It has a low variance in response time and waiting time.
- There is a very low risk of starvation.
- Disadvantages of SCAN Algorithm
- Requests just visited by the disk head will have a relatively longer waiting time.
- Sometimes, it may not be optimal because it causes the disk head to move to the end of the disk by default, whether or not there are requests to be serviced.

Circular Scan (C-SCAN) Algorithm

- The [C-SCAN algorithm](#) addresses the limitations of the SCAN algorithm. Unlike the SCAN algorithm that rescans requests that have been serviced on the first run, the C SCAN disk scheduling algorithm returns to the starting point without scanning (again) any requests in between and then begins to service requests from the other end. Suppose we have our requests in this manner: (100,180,40,150,90,15,190). Using the C-SCAN algorithm, here's how the requests are serviced:



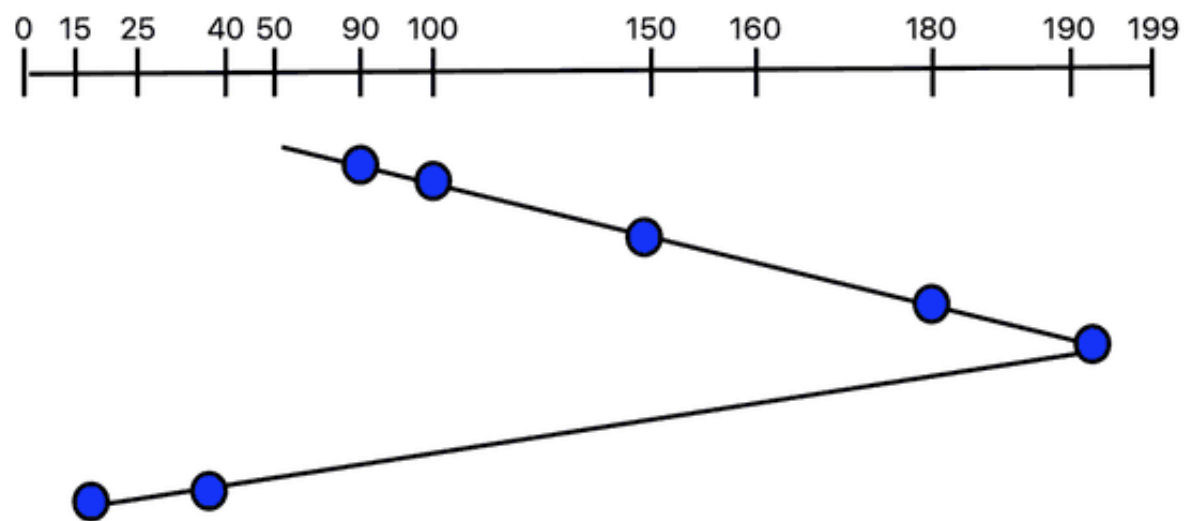
C-SCAN Algorithm

Pieces For Developers

- Here, we see that when the requests are serviced to the end point, it behaves similarly to the SCAN algorithm, but when it reverses its direction to the start point, it does not service any request until it gets to the start point; when it gets to the start point, it then services the request in the same direction.
- The total overhead movement is given by: $(90-50)+(100-90)+(150-100)+(180-150)+(190-180)+(199-190)+(199-0)+(15-0)+(40-15)= 388$.
Or simply: $(199-50)+(199-0)+(40-0)=388$
- Advantages of C-SCAN
- It provides a more uniform and accurate waiting time since all requests will be serviced once.
- It provides a better waiting time than the SCAN algorithm.
- Disadvantages of C-SCAN
- Like the SCAN algorithm, it causes the disk head to move to the end of the disk even if there are no available requests to be serviced.

LOOK Algorithm

- The [Look Disk Scheduling Algorithm](#) is similar to the SCAN algorithm. In the LOOK algorithm, the disk head moves to the last request (and not the end of the disk as is the case for the SCAN algorithm) to be serviced in front of the head, then it reverses its direction and services all the requests behind the disk head. This algorithm addresses the limitation of the SCAN algorithm by preventing the extra delay used in traversing to the end of the disk when there are no requests to be serviced. Let's see how the requests: (100,180,40,150,90,15,190) will be serviced using the LOOK algorithm:



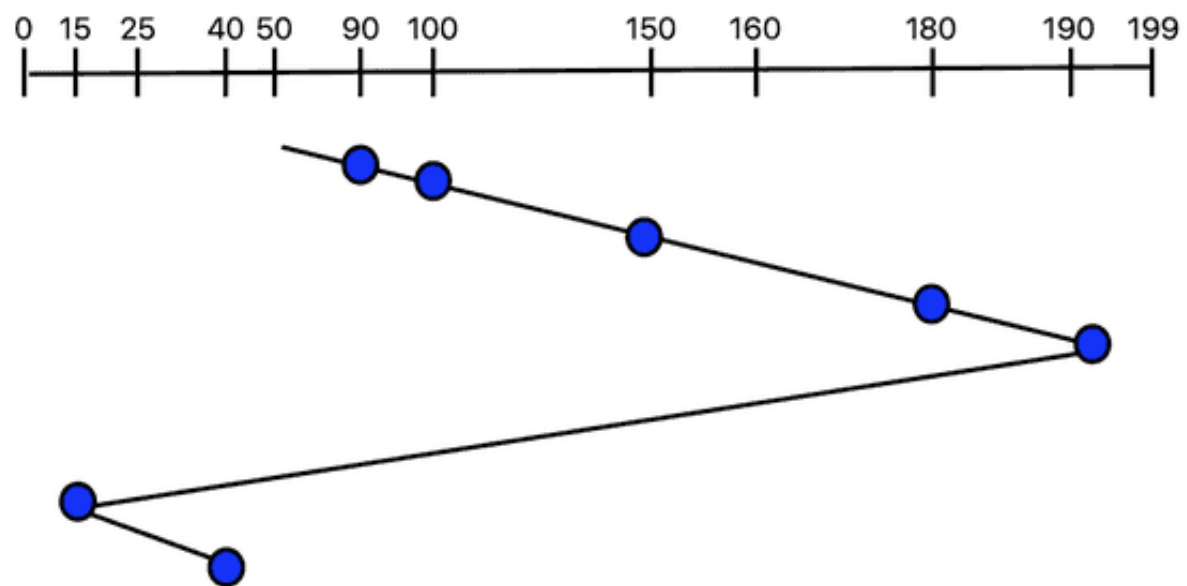
LOOK Algorithm

Pieces For Developers

- Here, we see that it traverses to track 190 (which is the furthest request to be serviced) and not track 199 (the end of the disk track). Then it reverses its direction and services the requests on track 40 and track 15.
- The total overhead movement is simply given by: $(190-50)+(190-15)=315$
- Advantages of LOOK Algorithm
 - It saves time that may have been wasted in unnecessarily traversing to the end of the disk when there are no requests.
 - It is efficient because only locations with data in it are visited.
 - It provides low variance in response time and waiting time.
- Disadvantages of LOOK Algorithm
 - There is an overhead in locating the end requests.
 - Requests that are located far away from the disk head are prone to starvation.

Circular Look (C-LOOK) Algorithm

- The [C-LOOK algorithm](#) is similar to the C-SCAN algorithm as it does not rescan requests that have already been visited on the first run to the end of the disk. The C LOOK disk scheduling algorithm allows the disk arm to reverse its direction and traverse to the first request (not the start point like the C-SCAN algorithm). Here's how a similar order of requests will be serviced using the C-LOOK algorithm:



- Here, just like the C-SCAN algorithm, it does not service any request (again) when it reverses its direction. The remaining requests are serviced after the disk arm reaches the start of the leftmost request.
- The total overhead movement is given by: $(190-50)+(190-15)+(40-15)=340$
- Advantages of C-LOOK Algorithm
- Compared to LOOK algorithm, it provides a better performance since it does not service requests that have already been serviced.
- Starvation of requests is avoided.
- It provides low variance in response time and waiting time.
- Disadvantages of C-LOOK Algorithm
- There is an overhead in finding end requests.